

HomeworkPlus

An AI-powered, multi-modal homework assistance platform — from photographed problem to step-by-step, spoken explanation.

Angular 19

FastAPI

NeonDB (PostgreSQL)

ChromaDB

LangChain / LangGraph

GPT-4o & Claude Sonnet

EasyOCR / PaddleOCR

Whisper STT & TTS

Redis + Celery

Docker Compose

Table of Contents

1. Project Overview	3
2. Technical Stack	4
3. System Architecture	5
4. Core Workflow — The Photo-to-Answer Pipeline	6
5. Real-Time Collaboration & Communication	8
6. Gamification & Engagement Engine	9
7. Architecture & Scalability	10
8. Risks, Assumptions & Next Steps	11

About this document

This Proof of Concept describes the design and workflow of HomeworkPlus, an AI-powered educational platform that turns a photographed homework problem into a guided, step-by-step learning experience through a coordinated multi-agent pipeline, real-time collaboration tools, and a gamified engagement layer.

1 Project Overview

HomeworkPlus is an advanced, AI-powered educational platform designed to provide interactive, multi-modal homework assistance. It goes beyond simple question-and-answer interactions by integrating a robust **"Photo-to-Answer" pipeline**, voice tutoring, real-time collaborative whiteboards, and a comprehensive gamified learning experience.

The platform is built around the idea that a student's learning moment starts the instant they get stuck — typically in front of a physical worksheet or textbook. HomeworkPlus removes the friction between "I don't understand this problem" and "I understand this concept" by allowing the student to simply photograph the problem and receive a structured, teaching-oriented explanation, with the option of a spoken walkthrough and live collaborative support.

1.1 Product Pillars

Photo-to-Answer Pipeline

A multi-agent system that converts an image of a homework problem into a structured, step-by-step explanation.

Voice Tutoring

Optional natural-sounding audio walkthroughs of any generated solution, powered by OpenAI TTS.

Real-Time Collaboration

Shared whiteboards and persistent chat sessions for continuous, contextual tutoring.

Gamified Engagement

XP, levels, achievements, and leaderboards to sustain motivation and daily usage.

1.2 Why a Multi-Agent Approach

Homework problems are heterogeneous: some are pure text, some are dense with mathematical notation, some include diagrams or graphs, and most require careful pedagogical framing rather than a bare final answer. A single monolithic model call struggles to do all of this reliably. HomeworkPlus instead routes each request through a pipeline of specialized agents — vision, privacy, OCR, math, retrieval, tutoring, and voice — each responsible for one well-defined transformation. This keeps the system debuggable, makes failure modes easier to isolate, and allows each stage to use the tool best suited to it (for example, OpenCV for image cleanup rather than asking an LLM to reason about pixels directly).

Target Outcome of this POC

Demonstrate that a photographed homework problem can be reliably converted into a grounded, pedagogically structured explanation — with acceptable latency — using an orchestrated pipeline of open-source and commercial AI components.

2 Technical Stack

The platform stack is organized into four layers: a rich interactive frontend, an asynchronous Python backend, a dual data layer (relational + vector), and a broad AI/ML ecosystem for perception, reasoning, and speech.

2.1 Frontend

Component	Purpose
Angular 19	Core application framework and routing
Angular Material	Consistent, accessible UI components
TailwindCSS	Utility-first styling for custom layouts
Three.js	3D learning maps for gamified topic progression
Fabric.js	Canvas engine powering the collaborative whiteboard

2.2 Backend

Component	Purpose
FastAPI	Asynchronous REST and WebSocket API layer
Python 3.12	Runtime for all backend services and agents
SQLAlchemy Async	Non-blocking ORM access to the relational database

2.3 Data Layer

Component	Role
NeonDB (PostgreSQL Serverless)	Primary relational store — users, sessions, progress, XP
ChromaDB	Vector database powering the RAG knowledge base
Redis + Celery	Caching layer and distributed task queue for async workloads

2.4 AI & Machine Learning Ecosystem

Category	Technologies
Foundational Models	GPT-4o, Claude Sonnet
Orchestration	LangChain, LangGraph
Vision & OCR	OpenCV, EasyOCR, PaddleOCR

3 System Architecture

The diagram below summarizes how the four layers of the stack relate to one another. The Angular frontend communicates with the FastAPI backend over REST and WebSockets; the backend in turn coordinates the data layer and the AI/ML ecosystem, offloading heavy workloads to Celery workers backed by Redis.

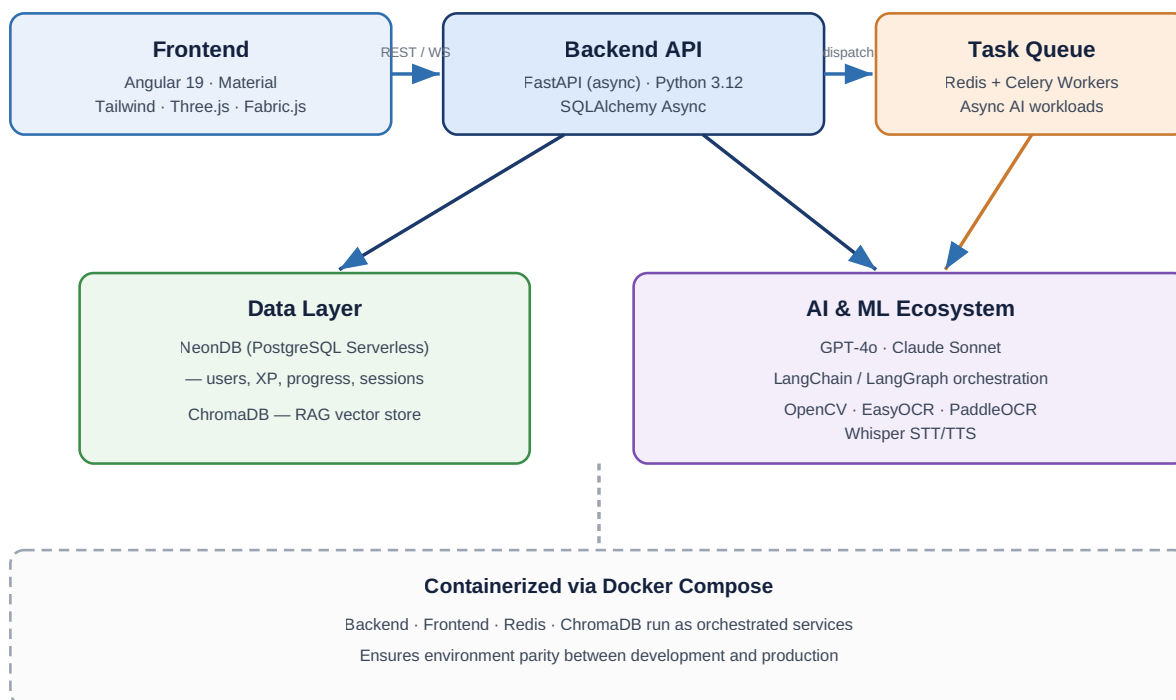


Figure 1 — High-level system architecture of HomeworkPlus.

3.1 Communication Patterns

The frontend communicates with the backend using two protocols: standard REST calls for stateless operations (uploading a photo, fetching a dashboard) and persistent WebSocket connections for stateful, real-time interactions such as whiteboard collaboration and continuous chat.

4 Core Workflow — The "Photo-to-Answer" Multi-Agent Pipeline

The centerpiece of this POC is a sophisticated orchestrator that routes tasks through specialized AI agents, transforming raw imagery into highly educational, step-by-step solutions. The full pipeline runs behind a single entry point, `POST /api/v1/ai/solve`, and is composed of seven internal stages before a response reaches the student.

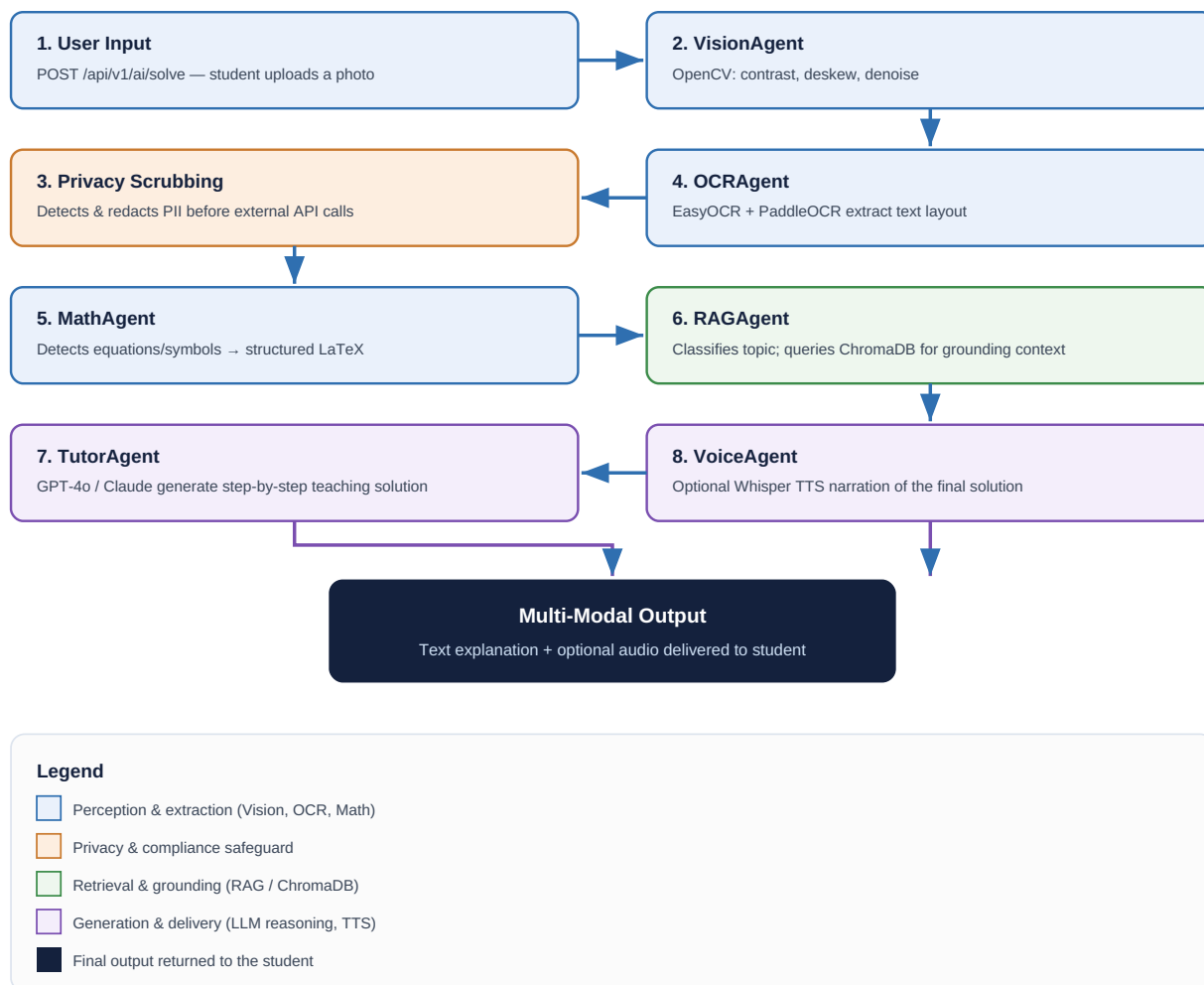


Figure 2 — The eight-stage Photo-to-Answer multi-agent pipeline.

4.1 Stage-by-Stage Detail

Step 1 · User Input

The student uploads a photo of their homework or problem set via `POST /api/v1/ai/solve`. This is the sole entry point into the pipeline, keeping the client-side integration simple regardless of how many agents run behind it.

Step 2 · Vision Pre-Processing (VisionAgent)

OpenCV is used to enhance image contrast, correct skew, and remove noise — standard pre-processing that measurably improves downstream OCR accuracy, especially for photos taken at an angle or in poor lighting.

Step 3 · Privacy Scrubbing

Before any data leaves the system boundary toward external APIs, the pipeline detects and redacts personally identifiable information present in the image (such as a student's name written on the page). This is treated as a mandatory gate rather than an optional step.

Step 4 · Text Extraction (OCRAgent)

EasyOCR and PaddleOCR run in tandem to extract the standard text layout of the page. Running both engines and reconciling their output improves robustness across handwriting styles and print quality.

Step 5 · Math & Symbol Detection (MathAgent)

Equations, graphs, and scientific symbols are identified and reliably converted into structured LaTeX, which is far easier for a downstream LLM to reason over accurately than raw OCR text.

Step 6 · Question Classification & Retrieval (RAGAgent)

The question is classified by subject and topic, and ChromaDB is queried to retrieve relevant contextual material — textbook formulas, similar solved examples, or syllabus context — grounding the eventual LLM response and reducing hallucination.

Step 7 · Educational Solution Generation (TutorAgent)

Using GPT-4o or Claude Sonnet, the system ingests the extracted text, LaTeX, and RAG context, and generates a structured, step-by-step explanation designed to teach the underlying concept rather than simply stating the final answer.

Step 8 · Multi-Modal Output (VoiceAgent)

The resulting solution is delivered as text and can optionally be converted into a natural-sounding audio explanation using OpenAI's TTS, supporting different learning preferences.

Real-Time Collaboration & Communication

Beyond the asynchronous pipeline, HomeworkPlus supports two persistent, bidirectional channels that keep the learning interaction alive after the initial solution is generated.

5.1 Collaborative Whiteboard

WS `/ws/whiteboard/{room_id}` enables real-time, bidirectional drawing and equation solving using WebSockets, allowing a student and a human or AI tutor to interact on the same canvas simultaneously. This is powered on the frontend by Fabric.js, giving a responsive drawing surface synchronized across participants in the same room.

5.2 Continuous AI Chat

WS `/ws/chat/{session_id}` provides a persistent WebSocket connection for multi-turn follow-up questions. This allows a student to ask clarifying questions about a generated solution without losing the surrounding context — the session retains the original problem, the extracted LaTeX, and the RAG context used to produce the first answer.

Why WebSockets

Both features require low-latency, bidirectional communication that a simple request/response REST call cannot provide efficiently. WebSockets keep a single open connection per room or session, avoiding the overhead of repeated polling.

5.3 Session Continuity

Because both channels are session- or room-scoped, a student can move from an initial photographed problem, into a clarifying chat, and onto a shared whiteboard with a tutor — all without re-establishing context. This continuity is central to the platform's pedagogical goal: it is not a single-shot answer engine but an ongoing tutoring surface.

6 Gamification & Engagement Engine

To maximize student retention and motivation, the platform features a deeply integrated gamification system layered on top of the core tutoring functionality.

6.1 Experience Engine (XP)

Users earn XP for interacting with the platform — solving questions, maintaining daily streaks, and engaging with follow-up chat or whiteboard sessions. XP is the base currency that drives all other gamification mechanics.

6.2 Progression System

XP accumulation drives dynamic leveling from level 1 to 100+, following a predefined XP curve. This gives students a long-horizon sense of progress that persists across many study sessions, independent of any single subject.

6.3 Achievements

Unlockable badges — such as "First Steps," "Math Wizard," and "Scholar" — reward specific milestones (first solved problem, sustained subject mastery, long streaks). Achievements are designed to recognize varied types of engagement, not just raw volume of questions solved.

6.4 Competitive Metrics

`GET /api/v1/dashboard/leaderboard` exposes global and subject-specific leaderboards, encouraging friendly competition among students. Subject-specific boards ensure that a student strong in one area (e.g. algebra) can still find a competitive niche even if they are behind on a global ranking.

Mechanic	Purpose	Primary Data Source
XP Engine	Reward day-to-day interaction	NeonDB — activity events
Levels 1–100+	Long-term progress signal	NeonDB — XP curve lookup
Achievements	Recognize varied engagement types	NeonDB — milestone rules
Leaderboards	Social, competitive motivation	NeonDB — aggregated rankings

7 Architecture & Scalability

7.1 Containerization

The entire stack — backend, frontend, Redis, and ChromaDB — is orchestrated via Docker Compose, ensuring environment parity across development and production. This significantly reduces "works on my machine" failure modes and simplifies onboarding for new contributors to the POC.

7.2 Asynchronous Processing

Heavy AI workloads — OCR passes, multi-step LLM chains, TTS generation — are decoupled from the main API thread and managed by Celery workers. This ensures the FastAPI backend remains highly responsive for lightweight operations (dashboard reads, session lookups) even while a computationally expensive pipeline run is in progress for another user.

7.3 Scalability Considerations

- **Horizontal worker scaling:** Celery workers can be scaled independently of the API layer, matching capacity to AI workload demand rather than raw HTTP traffic.
- **Serverless relational storage:** NeonDB's serverless PostgreSQL model allows the relational layer to scale with usage without manual provisioning.
- **Vector store isolation:** Running ChromaDB as its own containerized service keeps RAG retrieval performance isolated from relational query load.
- **WebSocket connection management:** Whiteboard and chat sessions are room/session-scoped, allowing connection state to be sharded across backend instances as usage grows.

Design principle

Every component that can fail slowly (an LLM call, an OCR pass, a TTS render) is pushed off the request/response path and into the async queue, so the user-facing API surface stays fast and predictable.

8 Risks, Assumptions & Next Steps

8.1 Key Assumptions

- Students will primarily submit photos taken with mobile devices under variable lighting and angle conditions, requiring robust pre-processing rather than assuming clean scans.
- Subject coverage in the initial POC is bounded to a defined curriculum scope, with the RAG knowledge base populated accordingly.
- External LLM providers (OpenAI, Anthropic) are assumed reachable with acceptable latency for near-real-time tutoring use cases.

8.2 Open Risks

Risk	Impact	Mitigation Direction
OCR/Math misreads on poor-quality photos	Incorrect grounding for the TutorAgent	Dual OCR engines + confidence thresholds with a re-capture prompt
PII leakage before scrubbing	Privacy / compliance exposure	Treat privacy scrubbing as a hard gate before any external API call
LLM hallucination on ungrounded topics	Incorrect teaching content	RAG grounding + confidence-based fallback to "insufficient context"
WebSocket scaling under concurrent load	Degraded whiteboard/chat responsiveness	Session-based sharding across backend instances

8.3 Suggested Next Steps

1. Build a thin vertical slice of the pipeline (Steps 1–7) for a single subject to validate end-to-end latency and answer quality.
2. Populate the ChromaDB knowledge base with an initial curriculum slice and measure retrieval relevance.
3. Pilot the whiteboard and chat WebSocket channels with a small group of students and human tutors.
4. Instrument the gamification layer with real usage data to validate the XP curve and achievement thresholds before wider rollout.